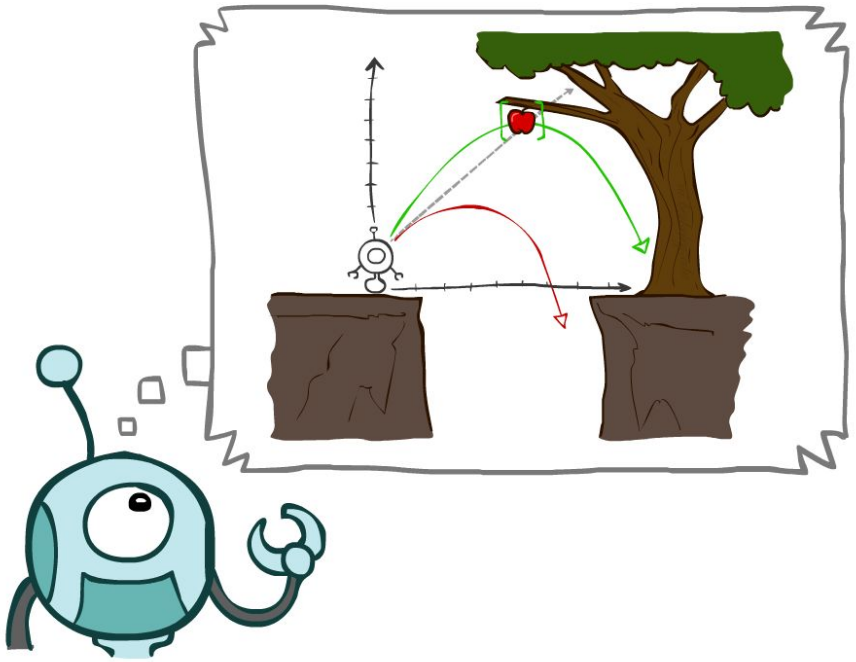
Intelligent Agents



Slide credits: AI BERKELEY

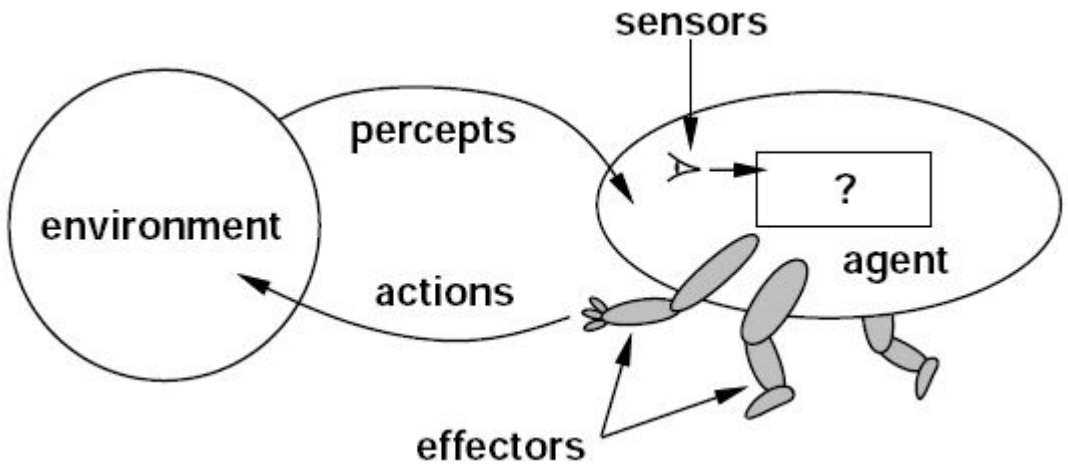
Agent

- An **agent** is **anything that can observe its environment, make decisions, and perform actions to achieve a goal.**
- In Artificial Intelligence, an agent does not have to be a robot. It can be:
  - A software program
  - A chatbot
  - A self-driving car
  - A robot
  - A game-playing AI

- The key idea is:
  - An agent perceives → thinks (reason) → acts.

What is an Agent?

- *Agent:*
  - Perceives (through sensors)
  - Acts (through actuators)
  - Reasons (chooses the best action)



- *"Intelligent agents continuously perform three functions: **perception** of dynamic conditions in the environment; **action** to affect conditions in the environment; and **reasoning** to interpret perceptions, solve problems, draw inferences, and determine actions."*

1. Percept

- **Percept** refers to the agent's perceptual inputs at any given instant.
  - What the agent sees ***right now.***
  - **Human Example**
    - Sensors include:
    - Eyes
    - Ears
    - Nose
    - Skin
    - Tongue
- AI Examples

Agent	**Sensors**
Self-driving car	Cameras, LiDAR, Radar, GPS
Robot vacuum	Infrared sensors, cameras
Chess AI	Current board position
ChatGPT	User's text prompt

Percept Sequence

- **Percept sequence:** the complete history of everything the agent has ever perceived
  - Everything it has seen ***so far***
- A self-driving car:
  - Percept = “Red light”
  - Percept sequence = “Green ⟶ Yellow ⟶ Red”
- The action depends on the whole percept sequence, not just current input.

2. Reasoning (think)

- After collecting information, the agent decides what to do, this is called **reasoning**.
- Reasoning may involve:
  - Applying rules
  - Searching
  - Planning
  - Learning
  - Comparing alternatives
  - Predicting outcomes
- The goal is to choose the **best possible action**.

- Example:
  - A GPS navigation system considers:
    - Distance
    - Traffic
    - Road closures
  - Then it selects the fastest route.

3. Action

- Finally, the agent performs an action using **actuators (effectors).**
  - **Human Actuators**
    - Hands
    - Legs
    - Voice
  - **Robot Actuators**
    - Wheels
    - Motors
    - Robotic arms
    - Speakers

Examples:

- Robot moves forward.
- ChatGPT generates a response.
- Self-driving car turns left.
- Vacuum cleaner changes direction.

Intelligent Agents

- An intelligent agent may learn from the environment to achieve their goals.
- Following are the main four rules for an AI agent:
  - **Rule 1:** An AI agent must have the ability to perceive the environment.
  - **Rule 2:** The observation must be used to make decisions.
  - **Rule 3:** Decision should result in an action.
  - **Rule 4:** The action taken by an AI agent must be a rational action.

Example of Agents

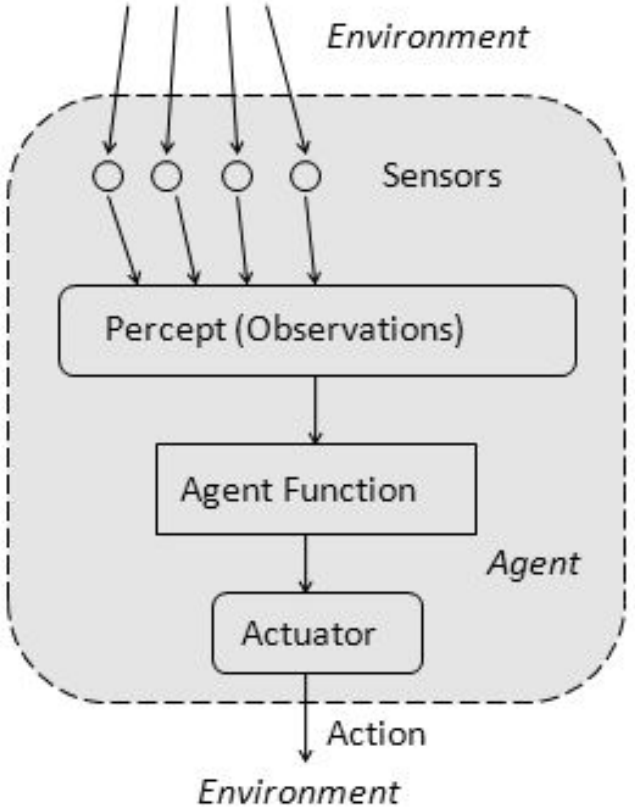
- Human Agent
  - eyes, ears, skin, taste buds, etc. for sensors
  - hands, fingers, legs, mouth, etc. for actuators
- Robot
  - camera, infrared, etc. for sensors
  - grippers, wheels, speakers, etc. for actuators
- Software Agent
  - functions as sensors
    - information provided as input to functions in the form of encoded bit strings or symbols
  - functions as actuators
    - results deliver the output

Rational Agent

- **Rational Agent**:
  - A **rational agent** is an agent that **chooses the action <u>expected</u> to produce the best outcome** based on:
    - What it has perceived so far (percept sequence).
    - What it already knows.
    - The actions available to it.
  - A rational agent simply makes the **best possible decision with the information currently available**.
- **Rationality** is distinct from **omniscience** (all-knowing with infinite knowledge)
  - **Actual (perfection) vs. Expected**
  - **Best vs. Optimal**

Agent Function and Program

- **Behavior of Agent –** It is the action that agent performs after any given sequence of percepts
- Agent's behavior is *mathematically* described by **Agent function**
  - A function mapping any given percept sequence to an action
- *Practically* it is described by **agent program**
  - The code that implements the agent function mapping
- Problems:
  - what is "the right thing"
  - how do you measure the "best outcome"



Specifying the Task Environment

Performance of Agents

Performance of Agents

- Before designing an AI system, we must define the task environment using PEAS.

- **PEAS:**
  - **Performance measure**
    - Defines how success of the agent is evaluated.
  - **Environment**
    - The external world in which the agent operates.
  - **Actuators**
    - The mechanisms through which the agent takes actions.
  - **Sensors**
    - The devices or inputs through which the agent perceives the environment.

PEAS: Automated Taxi

- Performance measure
  - Correct Destination, Fuel, time, cost, violations, safety, comfort, profit
- Environment
  - Roads, traffic
- Actuators
  - Steering, brake
- Sensors
  - Camera, radar, engine sensors

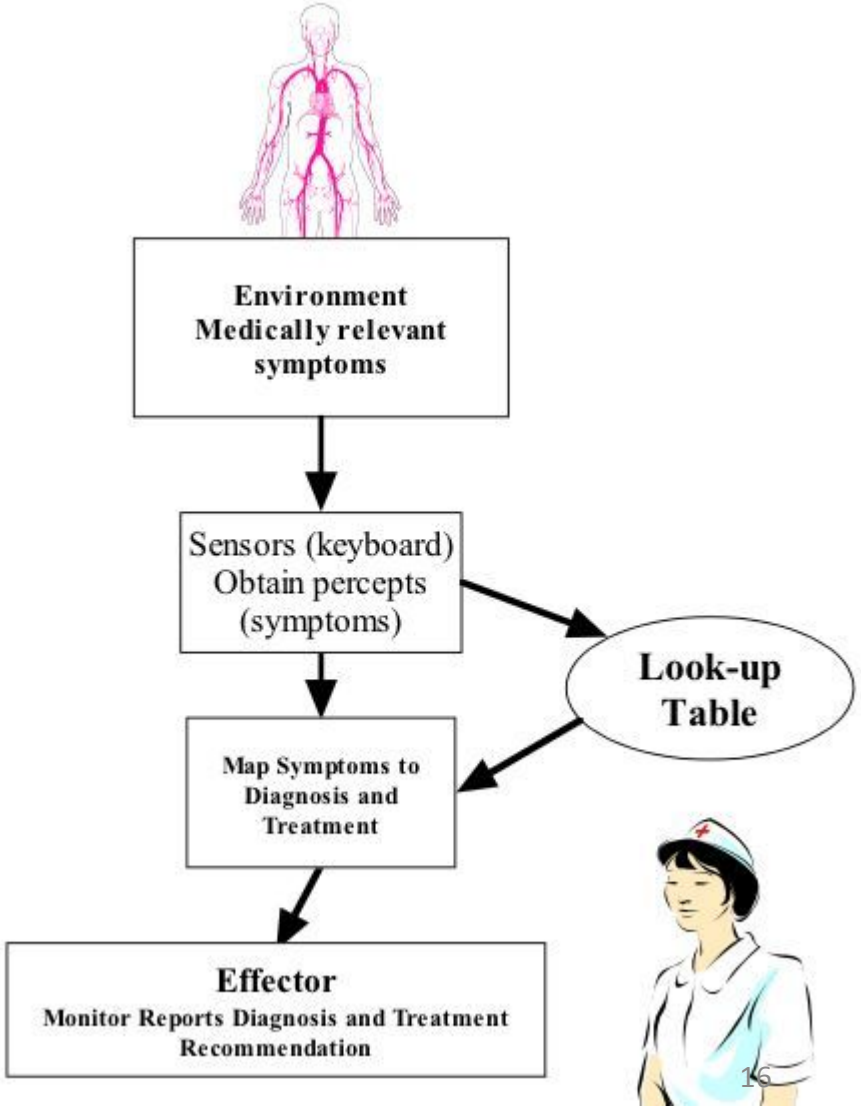


PEAS: Food Delivery Robot

- What would be the PEAS for a Food Delivery Robot?
- **Performance Measure:** Deliver correct food to the correct customer quickly, safely, and with minimal cost.
- **Environment:** Streets, sidewalks, traffic, pedestrians, weather conditions, restaurants, and customers.
- **Actuators:** Wheels/motors, robotic arm (if any), display/speaker for communication.
- **Sensors:** Camera, GPS, proximity sensors, obstacle detectors, microphone (optional).

PEAS: Medical diagnosis system

- Performance measure:
  - Healthy patient, minimize costs, lawsuits
- Environment:
  - Patient, hospital, staff
- Actuators:
  - Screen display (questions, tests, diagnoses, treatments, referrals)
- Sensors:
  - Keyboard (entry of symptoms, findings, patient's answers)



PEAS: Part-picking robot

- Performance measure:
  - Percentage of parts in correct bins
- Environment:
  - Conveyor belt with parts, bins
- Actuators:
  - Jointed arm and hand
- Sensors:
  - Camera, joint angle sensors





PEAS: Interactive English tutor

Activity

- Performance measure:
  - Maximize student's score on test
- Environment:
  - Set of students
- Actuators:
  - Screen display (exercises, suggestions, corrections)
- Sensors:
  - Keyboard

Environment Types

Properties of environments

- **Fully observable (Accessible)** vs. **partially observable (Inaccessible)**
  - **Fully Observable (Accessible):**
    - The agent can perceive the complete state of the environment at each time step.
  - **Partially Observable (Inaccessible):**
    - The agent cannot perceive the complete state of the environment.
  - Example:
    - **Chess** → Fully observable (you see entire board)
    - **Poker** → Partially observable (you don't see opponent's cards)
  - ***Is the current state of the world fully known at each step?***

Properties of environments

- **Deterministic** vs. **nondeterministic (Stochastic).**
  - Deterministic ⟶ next state can be determined
  - Stochastic ⟶ Randomness involved
  - If the next state of the environment is completely determined by the current state and the actions selected by the agents, then we say the environment is deterministic.
  - If the environment is inaccessible, then it may *appear* to be nondeterministic (bunch of uncertainties).

  Example:

  Chess ⟶ deterministic

  Taxi driving ⟶ stochastic

Environment Types

- **Episodic** vs. **Sequential**:
  - Episodic: If each decision is independent and does not influence future decisions, the environment is episodic.
  - The choice of action in each episode depends only on the episode itself.
  - Sequential: Current action affects future (Chess)

- **Example (Episodic): Email Spam Filter**
  - Each email is handled **independently**.
  - The agent reads one email (percept).
  - Classifies it as spam or not spam (action).
  - Moves to the next email.
  - The decision for one email **does not affect** the next one
- **Example (Sequential): Self-Driving Car**
  - If the car takes wrong turn now:
    → It affects arrival time
    → It affects fuel usage
    → It affects future route options

Environment Types

- **Static** vs. **Dynamic**:
  - If the environment can change while an agent is performing action, then we say the environment is dynamic for that agent; otherwise, it is static.
  - Dynamic environments are continuously asking the agent what it wants to do
  - Static environments are easy to deal with, because the agent does not keep on looking at the environment while it is deciding on an action.

  - Example:
    - Taxi → Dynamic
    - Crossword → Static

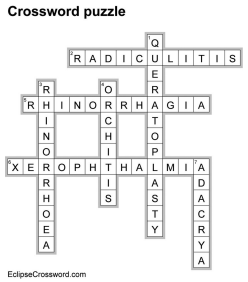
Environment Types

- **Discrete** vs. **continuous**:
  - Is there a finite (or countable) number of possible environment states?
  - The environment is discrete if the number of actions and possible states of the environment is finite otherwise it is continuous.
  - If there are a limited number of distinct, clearly defined percepts and actions, we say that the environment is discrete.
    - Chess, since there are a fixed number of possible moves on each turn.
    - Taxi driving is continuous.

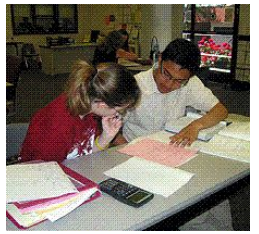
Types of Environment

- **Single Agent** vs. **Multi Agent**
  - Is the agent the only thing acting in the world?
  - In the single agent environment, there is only one agent
    - A computer software playing crossword puzzle
  - In multi-agent systems, there are more than one active agents
    - Video games

Environment Types Example





	Crossword puzzle	Chess	Taxi driving	Part-picking robot	English tutor
Observable	Yes	Yes	No	No	No
Deterministic	Yes	Yes	No	No	No
Episodic	No	No	No	Yes	No
Static	Yes	Yes	No	No	No
Discrete	Yes	Yes	No	No	Yes
Agents	Single	Multi	Multi	Single	Multi

Structure of Agents

Agent design

- **The environment type largely determines the agent design**
  - ***Fully/partially observable*** => agent requires ***memory*** (internal state)
  - ***Discrete/continuous*** => agent may not be able to enumerate ***all states***
  - ***Stochastic/deterministic*** => agent may have to prepare for ***contingencies***
  - ***Single-agent/multi-agent*** => agent may need to behave ***randomly***

Skeleton Agent Program

- Basic framework for an agent program

```
function SKELETON-AGENT(percept) returns action
    //The agent receives one percept from the environment and must return one action
  static: memory
    //memory is not reset every time, keeps past information, remembers history
  memory  := UPDATE-MEMORY(memory, percept)
    //the agent updates its internal state using old memory and new percept
  action  := CHOOSE-BEST-ACTION(memory)
    // the agent decides what to do, using its internal memory. This is where rationality happens.
  memory  := UPDATE-MEMORY(memory, action)
    //after choosing action the agent updates its memory again with the latest action.

return action
    //Finally the agent performs the action in the environment
```

Table Agent Program

- agent program based on table lookup

```
function TABLE-DRIVEN-AGENT(percept) returns action
  static: percepts  // initially empty sequence*
          table   // indexed by percept sequences
                  // initially fully specified

  append percept to the end of percepts
  action  := LOOKUP(percepts, table)

return action
```

* Note: the storage of percepts requires writeable memory

Structure of Agent

- All agents map percepts to actions, but they differ in intelligence level.
- Types of agents (increasing in generality and ability to handle complex environments)
  - Simple reflex agents
  - Model-based reflex agents
    - keep track of the world
  - Goal-based agents
    - work towards a goal
  - Utility-based agents
  - Learning agent

Simple Reflex Agents

Key Idea

- Act **only** on the current percept.
- Use **condition–action** (if–then) rules.
- **Do not maintain** any internal state or memory.
- Suitable for **fully observable** environments.
- Simple and fast, but limited intelligence.

How it Works

1. Receive the current percept from the environment.
2. Match it with a predefined rule.
3. Execute the corresponding action.

Architecture



Example: Automatic Door



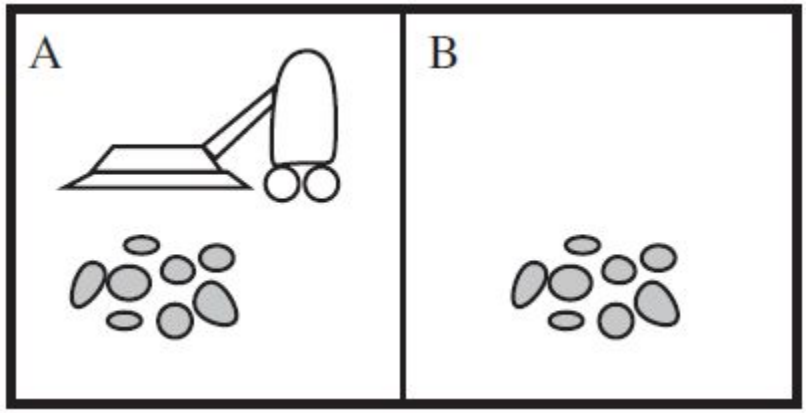
Percept
Motion sensor detects a person.

Rule
IF person detected THEN open door.

Action
Open the door.

Example: Vacuum Agent

- **Performance**?
  - 1 point for each square cleaned in time T?
  - #clean squares per time step - #moves per time step?
- **Environment**: vacuum, dirt, multiple areas defined by square regions
- **Actions**: left, right, clean, idle
- **Sensors**: location and contents
  - [A, dirty]



If status=Dirty then return clean
else if location=A then return Right
else if location=B then return Left

Simple Reflex Agent

function SIMPLE-REFLEX-AGENT(*percept*) **returns** an action
  persistent: *rules*, a set of condition–action rules

  state ← INTERPRET-INPUT(*percept*)
  rule ← RULE-MATCH(*state*, *rules*)
  action ← *rule*.ACTION
  return *action*

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.

```
def simple_reflex_agent(percept):
        state = get_state_from_percept(percept)
        rule = match_rule(state, rules)
        action = rule.Action
        return action
```

simple_reflex_agent.py hosted with ❤ by GitHub view raw

Problem with Simple Reflex Agent

- Many real-world environments are **partially observable**.
- This means the agent **cannot see the entire environment at all times**.
- Examples:
  - A self-driving car cannot see vehicles hidden behind a truck.
  - A robot cannot see what is behind a wall.
  - A vacuum cleaner cannot see dirt in another room.
  - A doctor cannot directly observe everything happening inside a patient's body.
- The agent therefore has **incomplete information**.

Model-Based Reflex Agent

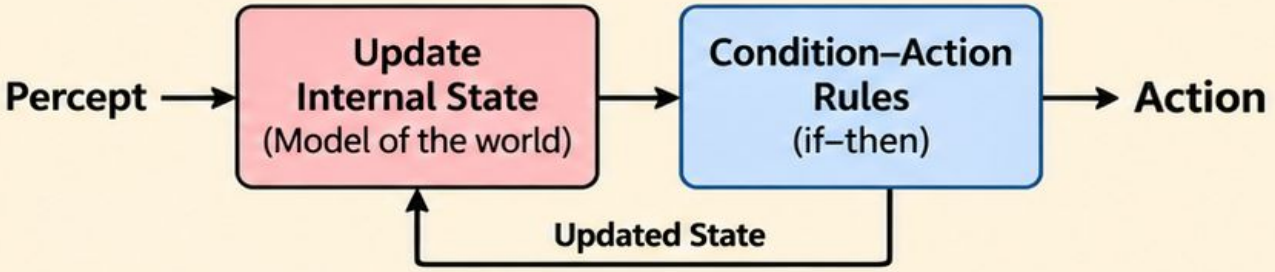
Key Idea

- Maintains an **internal model (state)** of the environment.
- Keeps track of **important information** that is not directly observable.
- Uses **condition–action rules** based on the current percept **and** internal state.
- Can handle **partially observable** environments.

How it Works

1. Receive a percept from the environment (e.g., sensor reading).
2. Update the internal state (model) based on the new percept and previous state.
3. Match the current percept and internal state with condition–action rules.
4. Execute the corresponding action.

Architecture



Example: Robot Vacuum Cleaner



Percept
Detects dirt, walls and obstacles using sensors.

Internal State (Model)
Maintains a map of the room, areas already cleaned and location of obstacles.

Rule
IF next cell is uncleaned and no obstacle THEN move forward.

Action
Move to the next location while avoiding obstacles.

Model-Based Agent

function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
  persistent: *state*, the agent's current conception of the world state
    model, a description of how the next state depends on current state and action
    rules, a set of condition–action rules
    action, the most recent action, initially none

  state ← UPDATE-STATE(*state*, *action*, *percept*, *model*)
  rule ← RULE-MATCH(*state*, *rules*)
  action ← *rule*.ACTION
  return *action*

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

```
1  def model_based_reflex_agent(percept):
2          state = update_state(state, action, percept, model)
3          rule = match_rule(state, rules)
4          action = rule.Action
5          return action
```

model_based_reflex_egent.py hosted with ❤ by GitHub view raw

Goal-Based Agents

Key Idea

- Have an **explicit goal** to achieve.
- Consider **possible actions** and choose those that **help achieve the goal.**
- Use **search and planning** to find a sequence of actions.
- More flexible than reflex agents because they can **plan ahead**.
- Suitable for **complex problem-solving** environments.

How it Works

1. Receive the current state from the environment.
2. Evaluate possible actions and predict their consequences.
3. Select actions that move the agent closer to the goal.
4. Execute the selected action and repeat until the goal is achieved.

Architecture



Example: GPS Navigation System



Percept (Current State)
Current location, road network, traffic conditions.

Goal
Reach the destination (Location B).

Possible Actions
Choose from multiple routes and next turns.

Action
Provide the next direction (e.g., turn right in 500 m).

Utility-Based Agents

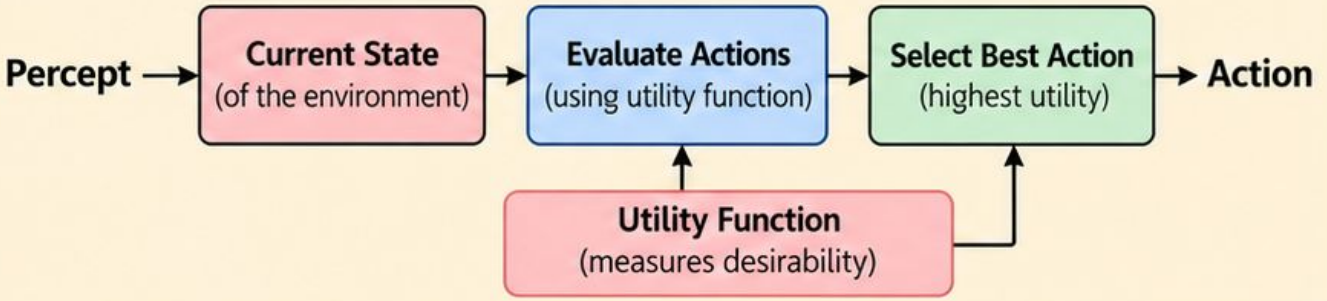
Key Idea

- When multiple actions can achieve the goal, a utility-based agent chooses the action that gives the **highest expected utility**.
- Uses a **utility function** to evaluate how desirable each possible outcome is.
- Considers **trade-offs** between different factors (e.g., time, cost, safety, comfort).
- More flexible and makes **better decisions** than goal-based agents.

How it Works

1. Receive the current state and possible actions.
2. Evaluate the expected utility (desirability) of each possible action using a utility function.
3. Select the action with the highest expected utility.
4. Execute the selected action.

Architecture



Example: Route Planning for a Ride-Hailing App



Percept (Current State)
Current location, destination, traffic conditions, road closures, etc.

Possible Actions
Take Route A, Route B or Route C.

Utility Function
Considers multiple factors such as travel time, traffic, fuel cost, tolls, and user preference (e.g., avoid traffic).

Action
Choose the route with the highest utility (e.g., Route B: 25 min with light traffic).

Learning Agents

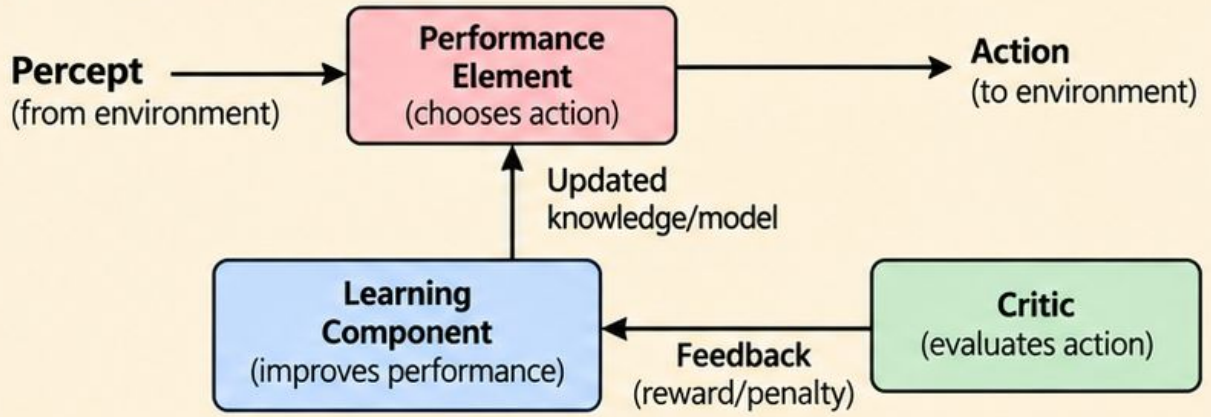
Key Idea

- Improve their performance through **experience and feedback**.
- Learn from **past interactions** instead of relying only on fixed rules.
- Use a **learning component** to update their knowledge/model.
- **Adapt** to changes in the environment.
- Can handle **complex and unknown environments**.

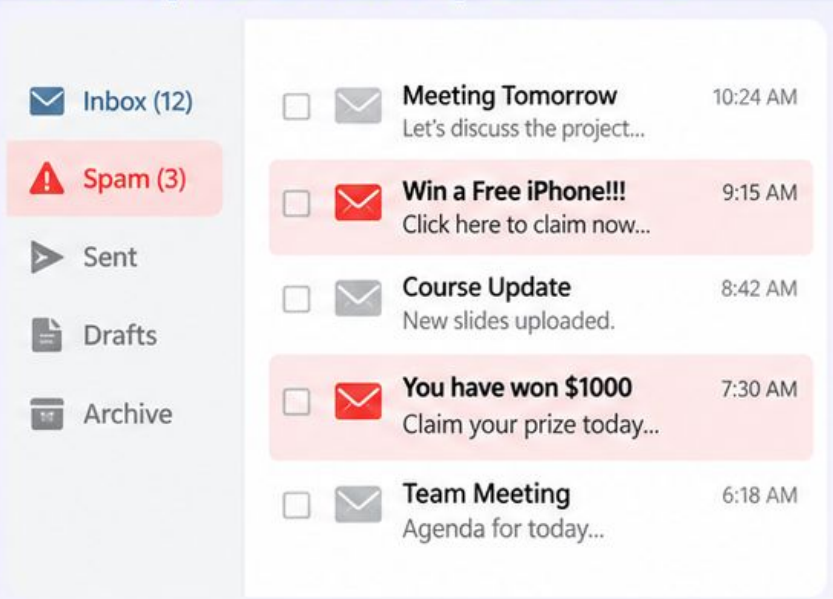
How it Works

1. Take perceps (observations) from the environment.
2. Use a learning algorithm to learn from experience and feedback.
3. Update the knowledge/model based on feedback.
4. Choose actions using the improved knowledge and continue learning.

General Architecture



Example: Email Spam Filter



Percept
Receive an email (its content, sender, subject, etc.).

Learning
Learn from past examples (labled emails) and user feedback (e.g., mark as spam/not spam).

Updated Model
Improved classification accuracy over time.

Action
Classify the email as Spam or Not Spam.

Intelligent Agents

- **Simple reflex agents** respond directly to percepts
- **model-based reflex agents** maintain internal state to track aspects of the world that are not evident in the current percept.
- **Goal-based agents** act to achieve their goals,
- **utility-based agents** try to maximize their own expected “happiness.”
- **Learning Agents** designed to act in uncertain environment